PC-2025/2026: Bigrams - Trigrams

Chiara Gori
E-mail address
chiara.gori9@edu.unifi.it
Matr. 7158493

Abstract

This report presents a sequential and parallel implementation of character-level and word-level n-gram histogram computation. The sequential algorithm was parallelized using OpenMP and tested with multiple scheduling policies and chunk sizes. Experiments were conducted on a corpus derived from the English novel Jane Eyre (retrieved legally through Project Gutenberg), which was multiplied 200 and 700 times. The best speedup achieved was 8.95x for word trigrams with dynamic scheduling and chunk size 64 using 14 threads, and 6.36x for character trigrams using 14 threads with static scheduling.

1. Introduction

In natural language processing and computational linguistics, an n-gram is a contiguous sequence of n items, either characters or words, extracted from a text. Computing n-gram frequency histograms is a computationally intensive task, as it requires iterating over millions of positions and updating shared data structures: clearly, it is a good candidate for parallelization.

The focus of this laboratory is to implement and evaluate parallel versions for bigrams and trigrams of both characters and words, through comparison of different OpenMP [2] scheduling policies and chunk size, with the goal of identifying the best configuration for each test case.

2. Implementation

The input text is preprocessed by reading the entire file into memory (instead of streaming or using intermediate files to reduce I/O overhead and improve cache locality) and applying a cleaning step. The function `cleanText()` discards non-ASCII characters, converts all alphabetic characters to lowercase, preserves digits and converts punctuation and (multiple) spaces in a single underscore character. The result is a clean string over an alphabet of 37 symbols: 26 lowercase letters, 10 digits and the underscore. This choice implies character n-grams spanning word boundaries (example: the bigrams of `'the cat'` are `th`, `he`, `e_`, `_c`, `ca`, `at`), which is a common approach in subword tokenization methods used in large language models. For word n-gram extraction, a `getWords()` function splits the cleaned string on the underscore to produce a list of individual words.

2.1. Sequential Implementation

2.1.1 n-grams of Characters

Character n-gram extraction works by sliding a window of size n over the cleaned string and mapping each n-gram to an integer index. The mapping, handled by a `encodeNgram()` function, uses direct addressing in base 37: digits map to offsets 0–9, lowercase letters to 10–35, and the underscore to 36. The resulting index uniquely identifies each n-gram and is then used as a direct index for a flat integer vector of size 37^n, i.e. 1,369 slots for bigrams and 50,653

for trigrams. This completely eliminates hashing overhead and provides $O(1)$ access with no collisions. The inverse mapping is handled by `decodeNgram()`, which reconstructs the original n-gram string from its integer index.

As a concrete example, consider the bigram `he` at position $p = 1$ in the string `the_cat`:

- $i = 0$: `c = 'h'` $\rightarrow$ `offset = 'h' - 'a'` $+10 = 17 \rightarrow$ `index` $= 0 \times 37 + 17 = 17$
- $i = 1$: `c = 'e'` $\rightarrow$ `offset = 'e' - 'a'` $+10 = 14 \rightarrow$ `index` $= 17 \times 37 + 14 = 643$

Thus, `histogram[643]++`, i.e. the bigram `he` is counted at index 643 of the histogram vector.

2.1.2 n-grams of Words

For word n-grams, consecutive groups of n words from the word list produced by `getWords()` are concatenated into string keys, whose count is incremented in an `unordered_map`.

```
for (int i = 0; i <= (int)words.size() - n;
     i++) {
    string key = words[i];
    for (int j = 1; j < n; j++)
        key += "_" + words[i + j];
    local[key]++;
}
```

2.2. Parallel Implementation

Both n-gram variants are parallelized using OpenMP. The parallelization strategy is based on chunk-based text processing, combined with the choice of instantiating a local histogram for each thread, which is updated independently with no need for synchronization. After the parallel region, the results are merged sequentially.

2.2.1 n-grams of Characters

For character n-grams, the main loop is parallelized with `#pragma omp parallel for schedule(static)`, distributing iterations evenly across threads: this choice is motivated by the uniform workload per iteration, making load unbalance unlikely. As stated above, each thread writes exclusively to its own local integer vector of size 37^n, so no synchronization is needed during computation. The merge step is a simple element-wise sum over all local vectors, which is fast due to the small and fixed size of the histogram.

The parallel extraction of character n-grams is implemented as follows:

```
#pragma omp parallel num_threads(numThreads
    )
{
    int id = omp_get_thread_num();
    vector<int>& local = localHists[id];
    #pragma omp for schedule(static)
    for (int p = 0; p <= (int)text.size() -
         n; p++) {
        int idx = encodeNgram(text, p, n);
        if (idx >= 0) local[idx]++;
    }
}
```

2.2.2 n-grams of Words

The implementation of word n-grams is less straightforward. Word length and hash map behaviour are variable: the workload per iteration is less uniform because string concatenation and hash map insertion have variable cost depending on word length and hash collisions.

Thus, the main loop is parallelized using the `#pragma omp parallel for` directive with three scheduling policies (static, dynamic and guided) and four chunk sizes, which are `auto`, 64 (small), 512 (medium) and 4096 (large). The chunk size controls the granularity of work distribution: smaller chunks improve load balancing at the cost of higher scheduling overhead, while larger chunks reduce overhead but may cause load imbalance.

Each thread maintains a private `unordered_map` and, after the parallel region, the local maps are merged sequentially into the global map by iterating over all keys of each local map. Merging hash maps requires string comparisons and dynamic memory operations (while merging integer vectors is a simple element-wise addition), which makes the word

n-grams merging step computationally heavier than in the case of character n-grams.

3. Experimental Setup

3.1. Hardware specifications

All experiments were conducted on a machine with the following specifications:

- CPU: Intel Core Ultra 7 155U
- Physical cores: 7
- Logical threads: 14
- RAM: 12 GB (allocated to WSL2)

3.2. Software Environment

Experiments were run on Ubuntu 22.04 under Windows Subsystem for Linux 2 (WSL2). The code was compiled with g++ 11.4.0 using the flags `-std=c++17 -fopenmp`.

3.3. Dataset

The corpus used for benchmarking is derived from the novel *Jane Eyre* by Charlotte Brontë, legally retrieved from Project Gutenberg [1]. The original file ($\sim$1 MB) was replicated to produce two corpora: a medium-sized corpus of approximately 200 MB (x200) and a large corpus of approximately 700 MB (x700). A corpus of 700 MB was selected to guarantee sequential execution times above 10 seconds for all tested configurations (bigrams and trigrams of both characters and words), as shorter runtimes can be dominated by parallelization overhead and fail to show parallelization benefits correctly. However, parallelization is also tested on a shorter medium-sized corpus to evaluate the effect of corpus size on parallel scalability.

3.4. Tested Parameters

Six thread counts were evaluated: 1, 2, 4, 8, 14 and 16. Tests cover the range from single-threaded execution to the upper bound of 16, which slightly exceeds the number of available logical threads (14), resulting in observable over-subscription.

As stated in section 2.2, Character n-gram extraction was only tested with static scheduling and automatic chunk size, while word n-grams were computed with three scheduling policies, i.e. static, dynamic and guided, combined with four chunk sizes: `auto` (OpenMP default), 64 (small), 512 (medium) and 4096 (large).

3.5. Measurement Methodology

To obtain stable measurements, each configuration was executed 7 times, of which the first 2 were discarded as warm-up runs to avoid cold cache effects. Performance metrics were computed over the remaining 5 runs: mean, standard deviation, minimum and maximum wall-clock time, as well as mean CPU time.

Wall-clock time was measured using `std::chrono::high_resolution_clock` inside the parallel function, excluding I/O, while CPU time was measured using `clock()` around the parallel function call, capturing the total processor time summed over all threads.

The main focus of this analysis is speedup, which is defined as the ratio between the sequential and parallel mean wall-clock times.

4. Results and Analysis

All data was collected into CSV files and all the plots below were generated with a Python script.

4.1. Correctness Validation

Before systemically exploring different thread counts and chunk sizes, the program computes a validation check: the correctness of the parallel implementation is verified by comparing the output of the parallel function with the sequential baseline on the original Jane Eyre corpus ($\sim$1 MB) using two functions, `validateChar()`, which consists in an element-wise comparison of histograms, and `validateWord()`, where two `unordered_map` instances are compared key by key. The validation code uses 4 threads and both bigrams and trigrams of characters and words pass the test.

4.2. Character n-grams

Figures 1 and 2 show the speedup curves for character bigrams and trigrams on the x200 and x700 corpora, respectively.

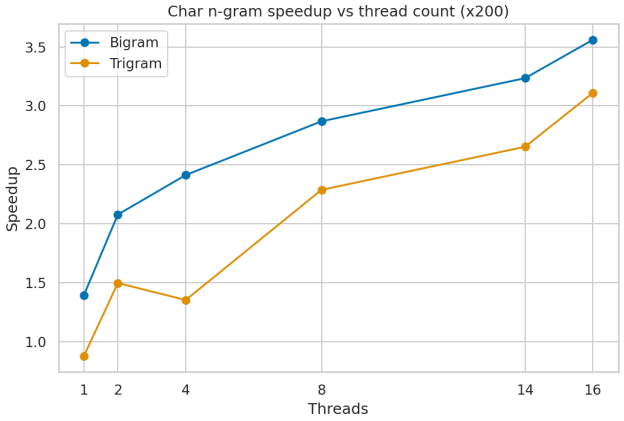

Figure 1. Char n-gram speedup vs thread count (x200).

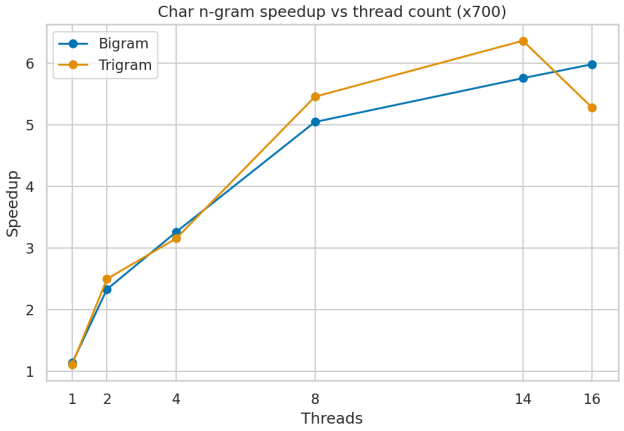

Figure 2. Char n-gram speedup vs thread count (x700).

On the large corpus (x700), both variants scale well up to 14 threads, reaching a peak speedup of 5.75x for bigrams and 6.36x for trigrams. The speedup at 16 threads decreases slightly for trigrams and increases marginally for bigrams, suggesting mild oversubscription effects near the physical limit of the machine.

On the smaller x200 corpus, the speedup is consistently lower compared to the bigger corpus, peaking at 3.56x for bigrams and 3.11x for trigrams at 16 threads. This difference is expected: with shorter sequential execution times, the fixed overhead of thread creation, local histogram allocation and the merge step represent a larger fraction of total execution time, limiting the achievable speedup.

4.2.1 Corpus Comparison

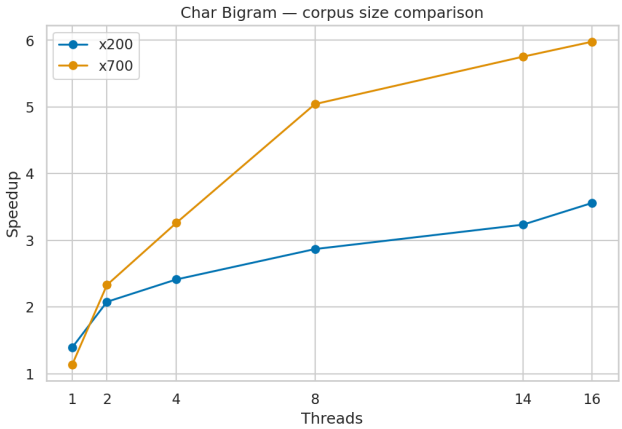

Figure 3. Char bigram corpus comparison.

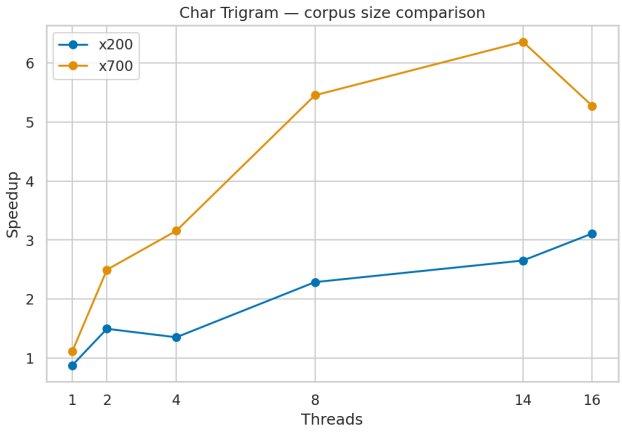

Figure 4. Char trigram corpus comparison.

Figures 3 and 4 directly compare the two corpus sizes. The gap between x200 and x700 is evident and consistent across all thread counts, confirming hard scaling behaviour: as the problem size grows, the parallel implementation becomes proportionally more efficient. This is consistent with Amdahl's Law: the sequential fraction (overhead and merge) is fixed and its relative cost decreases as the parallel workload grows.

4.3. Word n-grams

For brevity, the following analysis focuses on results obtained with the x700 corpus, which provides more stable and representative measurements. A corpus size comparison is presented in the paragraph 4.3.3.

4.3.1 Scheduling Policy

Figures 5 and 6 compare static, dynamic and guided scheduling with `auto` chunk size on the x700 corpus.

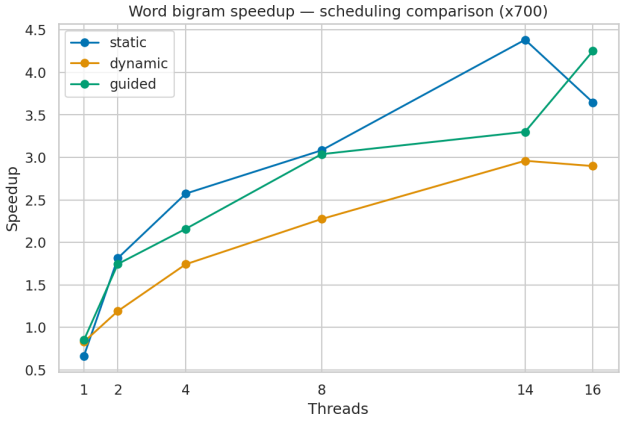

Figure 5. Word bigram speedup: scheduling comparison (x700).

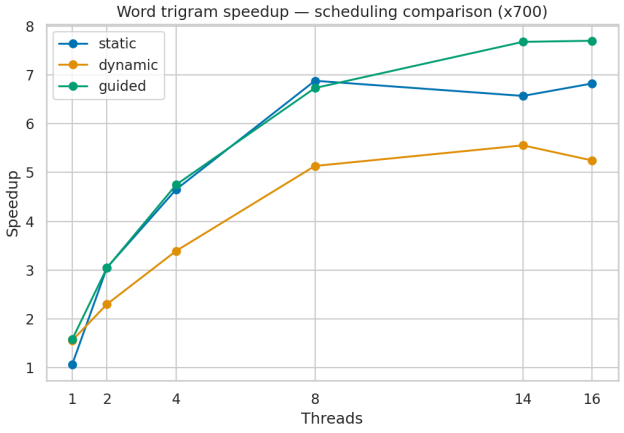

Figure 6. Word trigram speedup: scheduling comparison (x700).

For both bigrams and trigrams, static and guided scheduling policies provide similar results, with static performing better at 14 threads, and guided outdoing static at 16 threads. Dynamic scheduling consistently performs worse, which is explained by its very small default chunk size (chunk=1), which causes excessive scheduling overhead.

Static scheduling with 14 threads is the overall better choice for bigrams (4.38x speedup), while guided scheduling with 16 threads provides the best speedup for trigrams (7.67x).

4.3.2 Effect of Chunk Size

With reference to figure 7, bigrams show moderate variation across chunk sizes: `auto` performs best (4.38x at 14 threads), while chunk=512 and chunk=4096 are slightly worse. The difference is small because static scheduling with any chunk size distributes work evenly, so that the chunk size mainly affects memory access patterns.

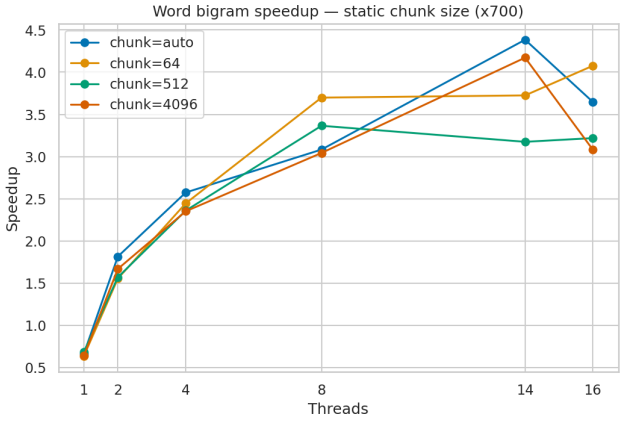

Figure 7. Word bigram speedup: static scheduling, chunk size comparison (x700).

As shown in figure 8, chunk sizes have very little effect also for trigrams with static scheduling, with chunk=64 with 14 threads providing the best speedup of 8.5x.

For dynamic scheduling (figures 9 and 10), the effect of chunk size is much more pronounced. In both cases of bigrams and trigrams `auto` (chunk=1) is consistently the worst performer, as explained above.

For bigrams, chunk=4096 achieves the best result (4.33x at 14 threads), followed closely by chunk=64 and chunk=512. For trigrams with dynamic scheduling, chunk=64 achieves the best overall result (8.95x at 14 threads). This suggests

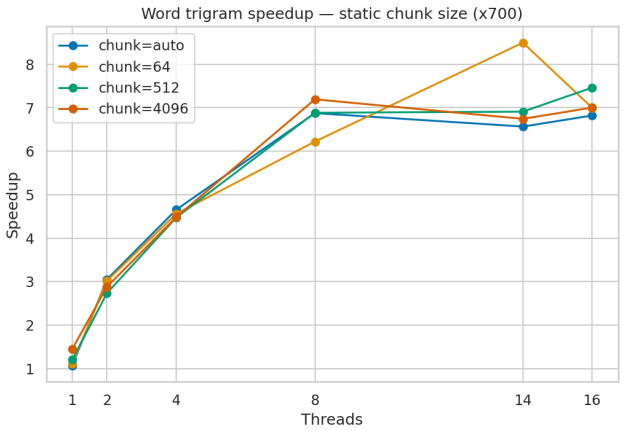

Figure 8. Word trigram speedup: static scheduling, chunk size comparison (x700).

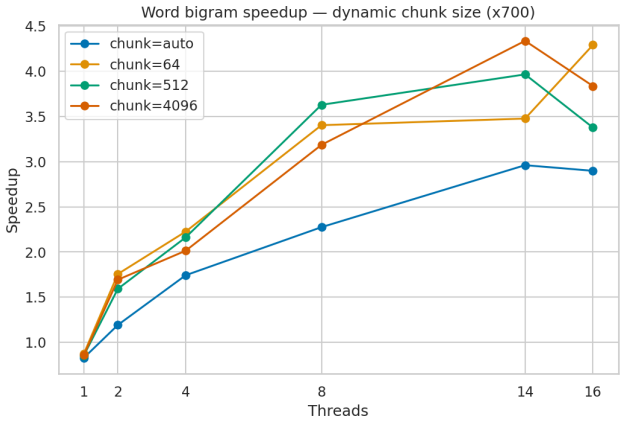

Figure 9. Word bigram speedup: dynamic scheduling, chunk size comparison (x700).

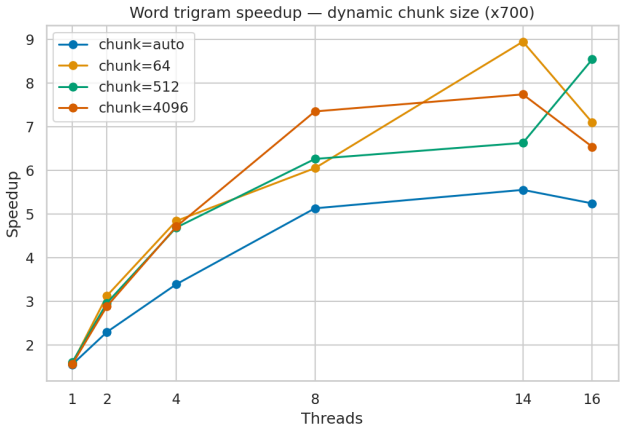

Figure 10. Word trigram speedup: dynamic scheduling, chunk size comparison (x700).

that for trigrams the slightly better load balancing of smaller chunks outweighs the scheduling overhead, while for bigrams the overhead dominates and larger chunks are preferable.

At 16 threads the ranking among chunk sizes changes (i.e. chunk=512 overtakes chunk=64 and chunk=4096 with a speedup of 8.54x), This can be explained by the interaction between chunk size and oversubscription: with more threads than logical cores, context switching can interrupt a thread mid-chunk, increasing scheduling overhead. chunk=64 suffers more from this effect because the large number of small chunks amplifies the cost of each interruption. chunk=4096 risks load imbalance when a thread is slowed down while processing a large chunk. chunk=512 represents the best trade-off in this scenario: it's large enough to amortize context switching cost but small enough to maintain load balance. This suggests that the optimal chunk size increases with oversubscription.

4.3.3 Corpus Comparison

Figures 11 and 12 below compare results obtained with a corpus of x200 and x700 the original novel. The following graphs use the results obtained with a static scheduling with chunk size=`auto`, which, as shown above, isn't necessarily the best configuration, but it allows a standard comparison across corpus sizes, and can be immediately compared with the comparison presented in section 4.2.1 for char n-grams.

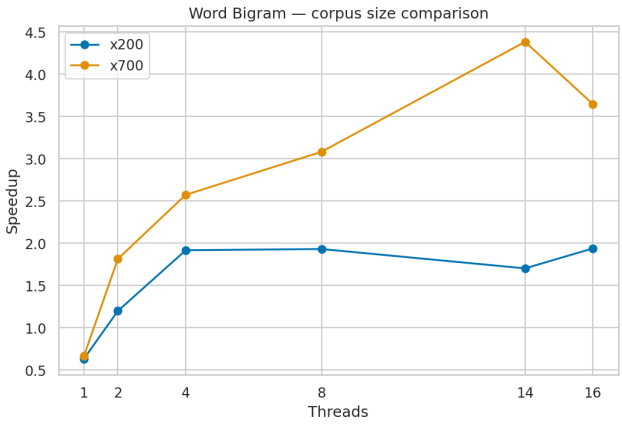

Figure 11. Word bigram speedup: corpus size comparison.

The gap is dramatic in both cases: the extrac-

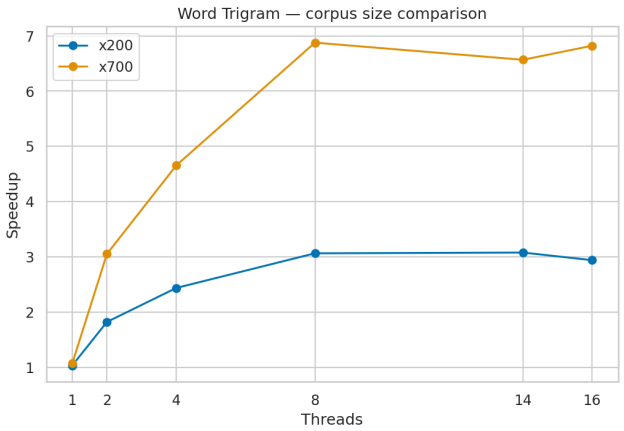

Figure 12. Word trigram speedup: corpus size comparison.

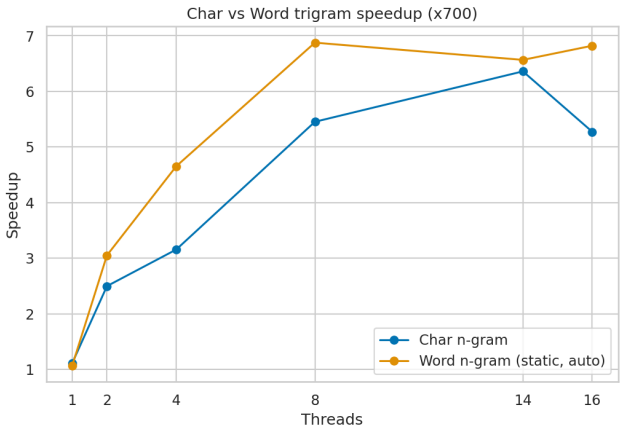

Figure 14. Speedup char vs word trigram comparison.

tion of bigrams and trigrams from the x200 corpus saturates around a speedup smaller than half of the speedup reached by the x700 corpus.

4.4. Char vs Word n-grams Comparison

Figures 13 and 14 directly compare the speedup of character and word n-grams, obtained with static scheduling and `auto` chunk size.

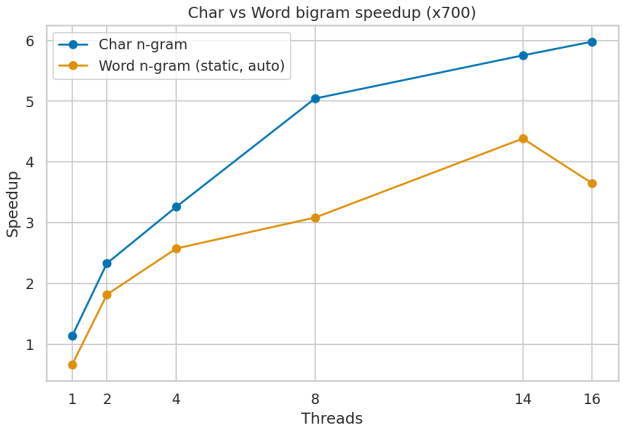

Figure 13. Speedup char vs word bigram comparison.

For bigrams, character n-grams consistently outperform word n-grams across all thread counts. This is expected, since character n-gram extraction benefits from a uniform workload and a lightweight merge step, while word bigrams suffer from the overhead of hash map operations and the costly sequential merge.

For trigrams, the comparison, surprisingly, reverses: word trigrams outperform character trigrams, although the gap is smaller than it was for bigrams. This reversal is explained by the higher per-iteration cost of word trigram extraction, as concatenating three words and performing a hash map insertion is significantly more expensive than encoding three characters in base 37: the heavier workload per iteration means that the parallel computation dominates over the merge overhead, leading to better speedup. This confirms the general principle that parallelization is more effective when the ratio of computation to synchronization overhead is high.

4.5. CPU Time Analysis

The figures presented in this paragraph show the CPU time as a function of thread count for all configurations, on the x700 corpus.

For character n-grams, i.e. graphs 15 and 16, the CPU time grows almost linearly with the number of threads up to 14, which indicates that all threads remain active throughout the parallel region with minimal idle time. At 16 threads the CPU time flattens, consistent with the oversubscription effect observed in the speedup curves.

A similar behaviour happens in word n-grams with static scheduling (figures 17 and 18 below), but a dip in CPU time can be observed at 2 threads. This is probably due to the fact that with only one parallel thread the overhead of allocating a private `unordered_map` and performing the merge is not amortized by any parallelization,

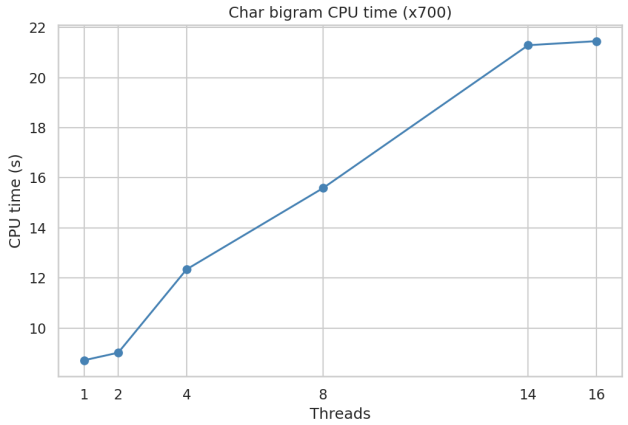

Figure 15. Char bigram CPU time (x700).

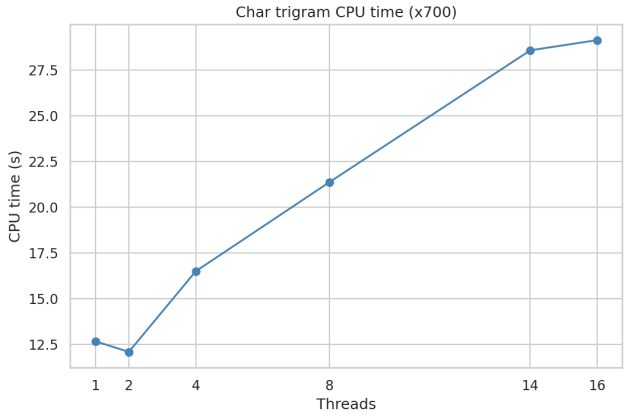

Figure 16. Char trigram CPU time (x700).

resulting in higher total CPU usage than with 2 threads where the work is split.

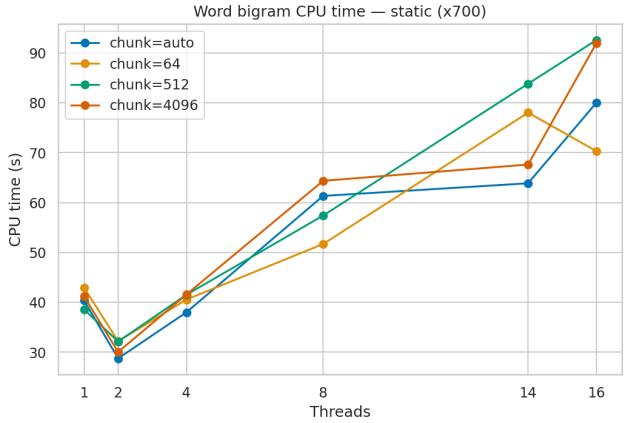

Figure 17. Word bigram CPU time: static scheduling (x700).

For word n-grams with dynamic scheduling (figures 19 and 20), the most striking observation is the much higher CPU time of chunk=`auto` compared to all other chunk sizes. Recalling that, for dynamic scheduling, `auto`=1, the OpenMP scheduler is invoked for each word in the corpus, and the resulting interaction cost dominates over the actual computation.

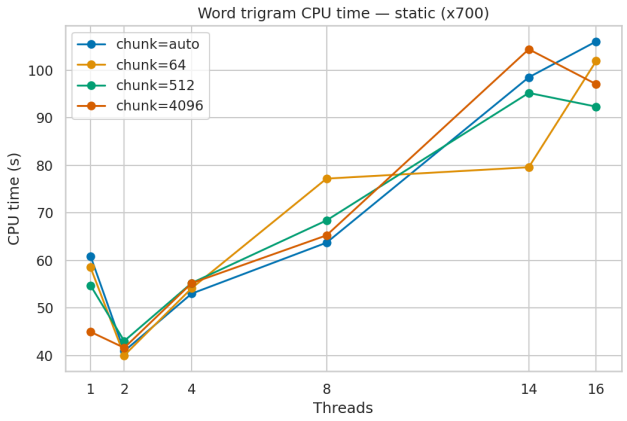

Figure 18. Word trigram CPU time: static scheduling (x700).

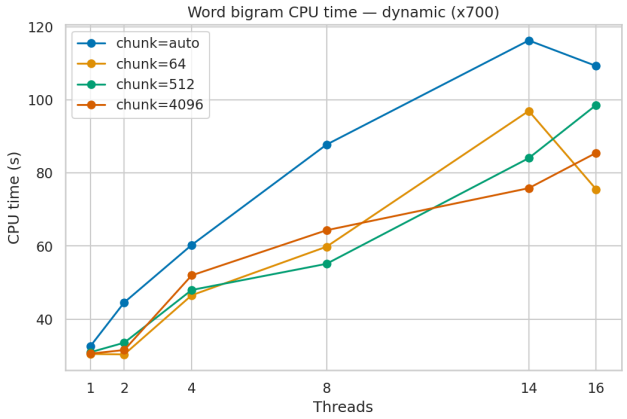

Figure 19. Word bigram CPU time: dynamic scheduling (x700).

5. Conclusions

This report presented a sequential and parallel implementation of character-level and word-level n-gram (particularly bigram and trigram) extraction, using OpenMP. The two implementations were compared to observe the relative speedup across all tested configurations.

For word n-grams, the best speedup achieved is 8.95x for trigrams with dynamic scheduling and chunk size 64 using 14 threads; for character n-grams the higher speedup is 6.36x for trigrams,

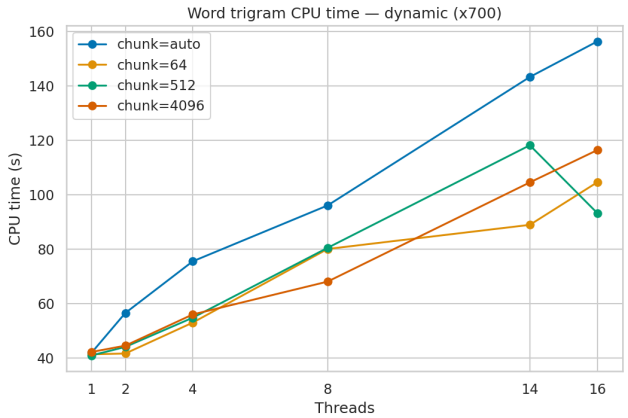

Figure 20. Word trigram CPU time: dynamic scheduling (x700).

using 14 threads with static scheduling.

Thus, the optimal configuration varies by the nature of the problem: character n-grams are best suited for static scheduling with `auto` chunk size as their workload per iteration is uniform, while word n-grams need a more careful selection of scheduling strategy and chunk size, but lead to higher speedups.

Additionally, the corpus size comparison confirmed that parallelization benefits increase with input size.

Future work could explore concurrent hash map implementations to parallelize the merge step for word n-grams and weak scaling analysis with variable corpus sizes proportional to thread count.

References

[1] C. Brontë. Jane Eyre, 1847.

[2] OpenMP Architecture Review Board. OpenMP specification.